A Comparative Study of Ligra and Ligra+: Shared-Memory Graph Processing with and without Compression

Shahzaib Khan Gakhar
RPTU Kaiserslautern

Abstract

Today's shared-memory machines offer hundreds of gigabytes to several terabytes of RAM, making it possible to process graphs with billions of edges on a single host.Many graph algorithm are till limited by memory speed rather than computation. Ligra raise this issue through a lightweight, data-parallel interface built around two primitives `VERTEXMAP` and `EDGEMAP` that dynamically change between sparse and dense traversal modes. Ligra+ extends this design with compression techniques such as delta encoding and compact variable-byte formats, reducing memory footprint while improving throughput on multi-core systems. This report shows the central ideas of both frameworks, reconstructs their algorithmic design principles, compare how they actually perform and analyzes how compression reshapes memory–computation trade-offs in shared-memory graph processing.

2012 ACM Subject Classification Theory of computation → Shared memory algorithms; Information systems → Graph-based data models

Keywords and phrases Ligra, Ligra+, shared-memory graph processing, compression, parallel algorithms

1 Introduction

Large graphs appear consistently in social networks, web infrastructure, biological interaction networks, and simulations of physical systems. While distributed graph-processing frameworks such as Pregel and GraphLab used to lead the field, modern servers now provide enough RAM to store ten billion edges or more on a single machine. Shared-memory systems reduce communication delays, simplify programming, and often work better than distributed solutions for many workloads [5].

Ligra takes advantage of this hardware trend by providing a minimal, efficient interface for data-parallel graph traversal. Rather than relying on message passing, Ligra organizes computations around dynamically changing *frontiers* sets of currently active vertices. The system automatically switches between sparse and dense evaluation modes depending on frontier size.

Ligra+ extends the same interface but compresses adjacency lists using delta-encoded and variable-length formats [6]. Earlier compression systems achieved space savings at the cost of slower runtime [1]. Ligra+ demonstrates conditions under which compression yields speedups instead, particularly under memory bandwidth pressure on multi-core machines.

This report explains the design choices behind Ligra and Ligra+, analyzes their performance characteristics, and compares how compression modifies algorithmic behavior in shared-memory graph workloads.

2 Background and Graph Model

Both Ligra and Ligra+ operate on directed graphs $G = (V, E)$ where vertices are numbered 0,to $|V| - 1$. For each vertex v, the tools store outgoing neighbors $N^+(v)$ and incoming neighbors $N^-(v)$ alongside their degrees [5]. Computation uses fork-join parallelism with

atomic instructions such as compare-and-swap (CAS) make sure results are correct when multiple writes happen at the same time.

Fork-join parallelism structures computation as a sequence of parallel tasks that are recursively spawned (forked) and later synchronized (joined). In Ligra, this model allows vertex- and edge-level operations to execute in parallel while ensuring synchronization at well-defined points, such as after each traversal step.

The fundamental abstraction is the `vertexSubset`, which represents an active frontier either as a sparse list of vertex identifiers or as a dense bitmap. The representation is selected dynamically based on frontier size.

Ligra provides two main parallel operations:

- **VERTEXMAP(U, F):** applies a Boolean function to each vertex in U in parallel and returns the subset of vertices for which the F returns true.
- **EDGEMAP(G, U, F, C):** processes edges from active sources in U, applies an update function F, and returns all target vertices that satisfy C and for which the update succeeds.

`EDGEMAP` automatically selects sparse traversal over outgoing edges or dense traversal scanning all vertices and their incoming edges, depending on the number of active edges.

Ligra+ preserves this API but replaces raw adjacency lists with compressed, delta-encoded representations and partitions large neighbor lists for better load balancing [6].

3 Ligra

3.1 Motivation

Ligra aims to provide a lightweight shared-memory alternative to distributed frameworks. Systems like Pregel [4] or GraphLab [3] cause extra time for coordination and data exchange, whereas Ligra is optimized for uniform memory access with fast, shared RAM.

3.2 Core Techniques

Ligra's performance depends on smartly switching between sparse and dense modes when frontier size crosses a threshold proportional to $|E|$ [5]. Sparse traversal examines only outgoing edges from active vertices, dense traversal checks all vertices using incoming edges.

Graphs are stored in flat arrays for both in-edges and out-edges, minimizing overhead. The `vertexSubset` abstraction simplifies algorithms such as BFS and Brandes' betweenness centrality [2].

Switching between sparse and dense traversal is crucial because the efficiency of each strategy depends on the size of the active frontier. When the frontier is small, sparse traversal is faster since it processes only the outgoing edges of active vertices. As the frontier grows large, sparse traversal can become inefficient due to repeated scanning of many adjacency lists, while dense traversal becomes faster by scanning all vertices once and stopping early when a valid predecessor is found. Using only a single traversal strategy would either waste work for small frontiers (dense-only) or incur excessive memory traffic for large frontiers (sparse-only), making hybrid traversal essential for high performance.

In BFS, the vertexSubset directly represents the current frontier, allowing each iteration to process only newly discovered vertices instead of scanning the entire graph. This abstraction cleanly separates traversal logic from data representation, enabling automatic switching between sparse and dense frontiers without algorithmic changes.

Consider a BFS step with a frontier containing vertices $\{v_1, v_2\}$. In Ligra, the adjacency lists of v_1 and v_2 are scanned directly to discover new neighbors. In Ligra+, the same traversal is performed, but the neighbor lists are first decoded from their compressed representation before processing. Importantly, the algorithmic structure and frontier evolution remain unchanged; only the cost of accessing edges differs.

3.3 Applications

Common algorithms implemented in Ligra include:

- Breadth-First Search (BFS)
- Betweenness centrality via Brandes' algorithm
- Connected components
- Graph radius approximation
- PageRank
- Bellman-Ford shortest paths

All implementations retain textbook asymptotic complexity (e.g., BFS in $O(|V| + |E|)$) while achieving near-linear speedup up to 40 cores [5].

4 Ligra+

4.1 Motivation

On modern systems, graph processing is frequently memory-bandwidth-bound: fetching adjacency lists dominates runtime relative to arithmetic. Ligra+ tests whether compressed representations decoded on the fly reduce bandwidth consumption enough to offset decompression costs.

4.2 Compressed Representation

Adjacency lists in Ligra+ are sorted and stored as sequences of delta values encoded using:

- variable-byte codes,
- run-length–coded byte sequences,
- nibble (4-bit) codes.

Run-length encoding groups values requiring equal byte lengths, reducing branching during decoding [6].

Decoding uses two core operations:

- **FirstEdge**: decodes the first neighbor.
- **NextEdge**: decodes subsequent neighbors from the last offset.

High-degree vertices are partitioned into chunks of size at most T, allowing parallel decoding with independent starting points, improving load balance on multi-core hardware.

4.3 Empirical Behavior

The reported performance results are based on experiments conducted on multi-core shared-memory machines with up to 40 hardware threads, using a mix of real-world and synthetic graphs with varying degree distributions. Ligra serves as the uncompressed baseline, and all speedups are measured relative to its execution time on the same hardware and datasets.

Across real and synthetic datasets, Ligra+ uses roughly half the memory of Ligra. Compression benefits scale with repeating structural patterns.

Single-threaded runtime is often slightly slower due to decompression. However, on 40-core machines, byte-coded Ligra+ achieves:

- up to 2× speedup,
- occasionally up to 10% slowdown,
- on average 14% faster than Ligra.

These gains appear when reduced memory traffic outweighs decoding overhead.

Among the compression schemes evaluated in Ligra+, run-length encoded byte codes generally provide the best performance trade-off. While nibble codes achieve slightly better compression ratios, their higher decoding cost often reduces throughput. Plain byte codes decode faster but offer less space reduction. Empirical results show that run-length encoded byte codes achieve close to the memory savings of nibble codes while maintaining decoding speeds comparable to byte codes, leading to the highest average speedups on multi-core systems.

5 Comparative Analysis

5.1 Abstraction and API

Ligra and Ligra+ share the same programming interface, any algorithm written for Ligra runs on Ligra+ unchanged. The key difference lies entirely in edge storage and traversal cost.

5.2 Space-Time Trade-offs

Ligra maximizes per-edge traversal speed but consumes more memory. Ligra+ cuts memory by nearly half, often resulting in faster performance for memory-bound workloads or large core counts.

For small graphs or lightly threaded execution, Ligra often wins. For large graphs or memory-constrained settings, Ligra+ is typically superior.

5.3 Algorithmic Sensitivity

Algorithms like BFS that perform little computation per edge benefit most from compression. Algorithms with heavier per-edge arithmetic, such as PageRank or Bellman-Ford, may hide decoding overhead, sometimes making Ligra+ faster in practice.

For example, during BFS, Ligra processes the current frontier by directly scanning adjacency lists in memory, whereas Ligra+ must first decode the compressed neighbor lists before traversal. Although this introduces additional decoding work, the reduced memory footprint often leads to fewer cache misses and lower memory bandwidth usage. As a result, BFS, which performs little computation per edge, can run faster in Ligra+ despite the added decoding step. In contrast, algorithms such as PageRank perform more arithmetic per edge, making decoding overhead comparatively less significant.

6 Conclusion

Ligra demonstrates that a simple shared-memory framework can support a broad class of graph traversal algorithms both efficiently and expressively. Ligra+ extends this model by showing that lightweight compression can simultaneously reduce memory usage and improve performance on modern multicore processors. Together, both systems highlight the importance of data layout, memory bandwidth, and threading decisions in designing high-performance graph processing systems.

Possible directions for `future` work include exploring adaptive compression schemes that dynamically change encoding formats based on degree distribution or runtime behavior. Another open question is how Ligra+'s compression techniques interact with emerging memory technologies such as high-bandwidth memory or non-volatile RAM. Finally, extending the framework to better support weighted and dynamic graphs remains an important area for further investigation.

References

1 Daniel K. Blandford, Guy E. Blelloch, and Ian A. Kash. Compact representations of separable graphs. In *SODA*, 2003.

2 Ulrik Brandes. A faster algorithm for betweenness centrality. *Journal of Mathematical Sociology*, 2001.

3 Yucheng Low et al. Graphlab: A new parallel framework for machine learning. In *UAI*, 2010.

4 Grzegorz Malewicz et al. Pregel: a system for large-scale graph processing. In *SIGMOD*, 2010.

5 Julian Shun and Guy E. Blelloch. Ligra: a lightweight graph processing framework for shared memory. In *PPoPP*. ACM, 2013.

6 Julian Shun, Laxman Dhulipala, and Guy E. Blelloch. Ligra+: graph processing with compression. In *Data Compression Conference*. IEEE, 2015.